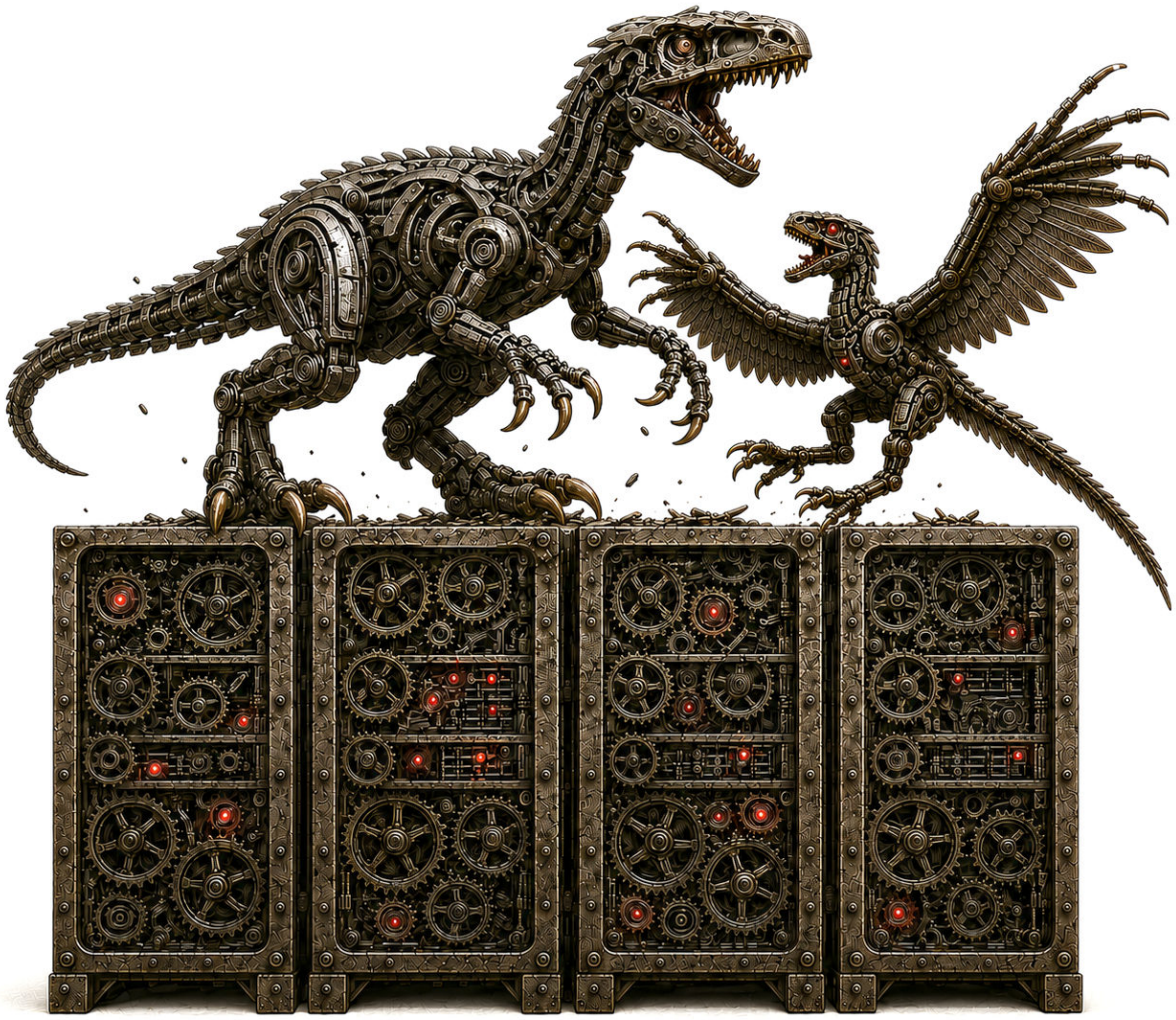


MEGARAPTOR and MICRORAPTOR
NSE Division HPC Clusters

User's Manual, ver. 2026-07-16

Division of Nuclear Science and Engineering, KTH

Jan Dufek

Contents

1 Specs **3**

2 Configuration **5**
2.1 Shared storage . . . 5
2.2 Slurm . . . 7
2.3 Modules . . . 11
2.4 Spack . . . 12
2.5 Conda . . . 13
2.6 Apptainer . . . 14

3 Computational software **16**
3.1 Monte Carlo solvers . . . 16
3.2 Cross section processors . . . 18
3.3 Thermal-hydraulics solvers . . . 18
3.4 Compilers and interpreters . . . 19
3.5 Machine Learning tools . . . 20

4 User accounts **22**
4.1 Secure access . . . 22
4.2 Request an account . . . 23
4.3 Logging in the cluster . . . 23
4.4 Practical info . . . 23

5 How to edit and debug input files **25**
5.1 Preparing Serpent input files . . . 25
5.2 Converting the input file format to Linux . . . 25
5.3 Manage your files on the cluster . . . 25
5.4 Debugging Serpent input file on the master node . . . 26

6 How to run simulations on compute nodes **27**
6.1 Which Slurm account you have . . . 27
6.2 Estimation of RAM allocation . . . 27
6.3 Running a simulation in the parallel mode . . . 28
6.4 Examples . . . 29
6.5 Monitoring your jobs . . . 31
6.6 Checking available nodes . . . 32

7 Terms and conditions for use **33**

1 Specs

> *I think there is a world market for maybe five computers.*
> Thomas Watson, chairman of IBM in 1943

The Division of Nuclear Science and Engineering (NSE) at KTH maintains two HPC clusters, the MEGARAPTOR and MICRORAPTOR, dedicated primarily for Monte Carlo simulations of nuclear reactors. Both clusters use the Slurm job scheduling system, and all software is installed in the form of Lmod modules, Spack and Conda packages, and Apptainer containers. Mastering the work on these clusters makes it easy for students to transition to supercomputers. Both clusters have been built, configured, maintained, administered and documented by Jan Dufek <jandufek@kth.se>, referred to as the "manager" in the rest of the text.

The MEGARAPTOR (**M**assively-parallel **E**nvironment for **G**eneral-purpose **A**dvanced **R**eactor **A**nalysis, **P**article **T**ransport, **O**ptimization, and **R**esearch) is a larger cluster, with nodes described in Tab. 1.1. The cluster contains computing nodes with modern CPUs with 332 CPU threads and 960 GB RAM, and it includes a node with the RTX 4090 GPU with the CUDA support. MSc students can get a group account on the MEGARAPTOR for the duration of their registration to the course Monte Carlo Methods and Simulations in Nuclear Engineering. The cluster is also used by MSc students working on degree projects, and it is also used for research.

Table 1.1: Specs of the MEGARAPTOR nodes

Node	P	CPU name	CPUs	RAM	O	L/S/M-Score
`x25`	-	Ryzen 9 9950X3D	32	128 GB DDR5	D	1.12 / 2.18 / 35.8
`n42`	p4	Ryzen 9 9950X3D	32	96 GB DDR5	D	1.12 / 2.18 / 35.8
`n41`	p4	Ryzen 9 9950X3D	32	96 GB DDR5	D	1.12 / 2.18 / 35.8
`n40`	p4	Ryzen 9 9950X3D	32	64 GB DDR5	D	1.12 / 2.18 / 35.8
`n32`	p3	Core i9 13900K	32	64 GB DDR5	D	0.97 / 2.11 / 31.2
`n31`	p3	Core i9 13900K	32	64 GB DDR5	D	0.97 / 2.11 / 31.2
`n30`	p3	Core i9 14900K	32	64 GB DDR5	D	1.00 / 2.16 / 32.1
`n22`	p2	Ryzen 9 7900	24	32 GB DDR5	Q	0.95 / 1.83 / 22.9
`n21`	p2	Ryzen 9 3950X	32	128 GB DDR4	D	0.57 / 0.89 / 18.1
`n20`	p2	Core i9 7940X	28	128 GB DDR4	D	0.57 / 1.18 / 15.8
`n01`	gpu	Ryzen 9 5900X [1]	24	96 GB DDR4	Q/D	0.79 / 1.56 / 18.9

The MICRORAPTOR is a smaller clusters, described in Tab. 1.2. While this cluster has only several computing nodes (and not modern ones), it has a relatively powerful master node x18. For this reason, its master node is programmed to also accept Slurm jobs when the regular computing nodes are overloaded (it can dedicate up to 24 CPU cores and 40 GB RAM for running Slurm jobs). In total, this cluster has 124 CPU threads and 448 GB RAM. Upon request,

[1] The `n01` node includes RTX 4090 gpu card.

MSc students can get a personal account on the MICRORAPTOR for the whole duration of their study at KTH, and they can use this cluster for assignments in the Gen IV and SMR courses or their self-improvements of using the computational software installed on the cluster.

Table 1.2: Specs of the MICRORAPTOR nodes

Node	P	CPU name	CPUs	RAM	O	L/S/M-Score
`x18`	p1	Threadripper 2990WX	64	128 GB DDR4	D	0.45 / 1.00 / 28.6
`n14`	p1	Core i7 8700K	12	64 GB DDR4	Q	0.64 / 1.26 / 7.7
`n13`	p1	Core i7 4930K	12	64 GB DDR3	D	0.45 / 0.89 / 5.3
`n12`	p1	Core i7 3930K	12	64 GB DDR3	D	0.44 / 0.82 / 5.3
`n11`	p1	Core i7 3930K	12	64 GB DDR3	D	0.44 / 0.82 / 5.3
`n10`	p1	Core i7 5820K	12	64 GB DDR4	D	0.46 / 0.89 / 5.3

In Tables 1.1 and 1.2,

- `x25` and `x18` are the master (login) nodes, connected to the internet via a public IP address,
- `n01` is a GPU computing node, all other nodes are CPU computing nodes,
- "P" denotes the Slurm partition to which the node belongs to,
- "CPUs" is the number of logical CPU cores (corresponding to the number of CPU threads),
- "O" denotes the provider of the server/GPU (having the right to remove the node from the cluster),
- "L-Score" is a single-thread score of the CPU under full stress (employing all of its threads fully), relative to performance of the x18 CPU running only a single process (a larger score means better performance),
- "S-Score" is a single-thread score of a CPU running a only single process, relative to performance of the x18 CPU running a single process,
- "M-Score" is a multi-thread score of a CPU under full stress, relative to performance of the x18 CPU running a single process. All scores reflect the wall-clock time of Serpent 2 simulations.

2 Configuration

> *It would appear that we have reached the limits of what it is possible to achieve with computer technology, although one should be careful with such statements, as they tend to sound pretty silly in 5 years.*
> John Von Neumann (stated in 1949)

2.1 Shared storage

All of the directories listed below have been set up on the master nodes of both clusters, and are visible to all computing nodes via NFS (Network File System).

2.1.1 `/users`

Your home account is created on the master node in the `/users` directory or its subdirectory. MSc students have a quota of 100 GB on the accounts. If you require additional space, please request it from the manager. Regardless of which computing nodes you use for your simulations, your output files will always be saved in your account directory so long as you run your calculations from within it. If you perform a calculation outside your home directory, the code will not be able to write the output files there and will exit with an error message. Your home directory is the only directory in which you have the permissions to create and modify files.

On MEGARAPTOR, the `/users` directory hosts all user accounts on 3 X 2TB NVMe drives. Additional 2 X 22 TB HDDs configured in a RAID 1 array (data mirrored on each disk) are available upon request for hosting files in special projects.

On MICRORAPTOR, the `/users` directory hosts all accounts on 2 X 17 TB HDDs configured in a RAID 1 array.

Although your data are relatively safe on our clusters, you are responsible for backing up all your important input and output files regularly. Your data may get lost due to fire, theft, hacker attacks or the manager error while re-configuring the cluster under root priviliges. Your data may also become unavailable to you when the cluster goes offline for whatever reason (which does happen from time to time, sometimes for several days). If you accidentally delete your data then you are in bad luck - our clusters don't back up any data.

2.1.2 `/xsdata`

The `/xsdata` directory contains the nuclear data libraries in recent and past versions and from several providers. You can use the data in these directories for your simulations directly, without copying them to your home directory. Therefore, copying the nuclear data libraries to your home directory is not allowed, as this fills up the disk space unnecessarily. All libraries in this directory are public and can be freely downloaded from the Internet.

Simulations that use the newest libraries tend to run slower and require (much) more RAM. Unless you require the most accurate results for a specific purpose (e.g. providing a safety analysis to the state authorities), it is almost always more reasonable to use older libraries, as this allows you to significantly reduce the RAM allocation and speed up your simulations.

Nuclear data libraries are present in `/xsdata` in the ACE and HDF5 formats. The Serpent code uses ACE libraries, while the OpenMC code uses HDF5 libraries. ACE-formatted libraries are usually prepared using the NJOY code from the base evaluated data in the ENDF-6 format (text-based pointwise data). The HDF5-formatted libraries are usually converted from the corresponding ACE libraries, but they can also be generated directly from the base ENDF-6 libraries. The cluster ACE and HDF5 libraries are listed in Tables 2.1 and 2.2.

Table 2.1: ACE libraries available on the cluster

Library	Path to the directory file	Depletion files
ENDF/B 7.0	`/xsdata/ACE/endfb7/endfb7.xsdata`	present
ENDF/B 7.1	`/xsdata/ACE/endfb7.1/endfb71.xsdata`	present
ENDF/B 8.1	`/xsdata/ACE/endfb8.1/endfb81.xsdata`	not present
JEFF 3.11	`/xsdata/ACE/jeff311/jeff311.xsdata`	present
JEFF 3.3	`/xsdata/ACE/jeff33/jeff33.xsdata`	present
JEFF 4.0	`/xsdata/ACE/jeff4/jeff4.xsdata`	not present

Table 2.2: HDF5 libraries available on the cluster

Library	Path to the directory file	Depletion files
ENDF/B 7.1	`/xsdata/HDF5/endfb7.1/cross_sections.xml`	present
ENDF/B 8.0	`/xsdata/HDF5/endfb8.0/cross_sections.xml`	present
ENDF/B 8.1	`/xsdata/HDF5/endfb8.1/cross_sections.xml`	present
JEFF 3.2	`/xsdata/HDF5/jeff3.2/cross_sections.xml`	not present
JEFF 3.3	`/xsdata/HDF5/jeff3.3/cross_sections.xml`	not present
JEFF 4.0	`/xsdata/HDF5/jeff4.0/cross_sections.xml`	present
JENDL 5.0	`/xsdata/HDF5/jendl5/cross_sections.xml`	not present
FENDL 3.2	`/xsdata/HDF5/fendl3.2/cross_sections.xml`	not present

The **ENDF/B** (Evaluated Nuclear Data File, version B) library was first released in 1968 and is developed and maintained by the National Nuclear Data Center (NNDC) at Brookhaven National Laboratory through the collaborative efforts of the Cross Section Evaluation Working Group (CSEWG).

The **JEFF** (Joint Evaluated Fission and Fusion) libraries are comprehensive nuclear data sets originating from a collaborative European effort, these libraries are managed by the Nuclear Energy Agency (NEA) Data Bank of the OECD. JEFF libraries serve as a critical alternative to US-based ENDF/B files.

The **JENDL** (Japanese Evaluated Nuclear Data Library) libraries are comprehensive compilations of evaluated nuclear data, developed and maintained by the Japan Atomic Energy Agency (JAEA), evolving from the work of the former Japan Atomic Energy Research Institute (JAERI).

The **FENDL** (Fusion Evaluated Nuclear Data Library) libraries are specialised nuclear data collections managed by the International Atomic Energy Agency (IAEA) to support Monte Carlo

simulations for fusion energy systems. Unlike general-purpose libraries like JEFF, which are heavily optimised for the thermal and resonance energy ranges characteristic of fission reactors, FENDL is specifically tailored for the high-energy neutron environment of fusion. It addresses the unique energy dependence of fusion systems by prioritising data accuracy around the 14 MeV neutron peak and extending evaluations to higher energies.

2.1.3 `/share`

All computational software is installed in the `/share` NFS directory and is therefore visible to the computing nodes. However, you can also execute any software installed in your home directory, such as Miniconda or executable files compiled from source code.

2.2 Slurm

Slurm (Simple Linux Utility for Resource Management) acts as the 'traffic controller' of an HPC cluster. As many users share the same hardware, Slurm ensures that everyone gets the resources they need (e.g. CPUs, RAM and GPUs) without crashing the system or interfering with others. You can learn how to work with Slurm using one of the many online sources, e.g. from instructions compiled for users of Dardel (#1 supercomputer in Sweden operated by the PDC Center for High Performance Computing at KTH): https://support.pdc.kth.se/doc/

Here is a short introduction to Slurm terminology, commands and parameters.

Slurm job
A "job" is one submission of **one or several** computational tasks (calculations) to Slurm, usually via the `sbatch` script file (see below). Upon submission of the `sbatch` script, Slurm will create one `JobID` number that identifies the job. The `sbatch` includes parameters that allocate resources on the cluster for the specific job; these resources will be blocked from all other users and jobs, and it is therefore important not to allocate resources that are not necessary for the specific job.

Slurm task
A "task" in Slurm terminology means **one independent process (calculation) that Slurm may start**.

Slurm partitions
In Slurm terminology, a 'partition' is a set of nodes that you may submit a job to. A partition usually represents a collection of nodes that have similar specs. When Slurm receives a request to execute a job on a specific partition, it checks whether any of the partition nodes has the requested resources. If so, the job is executed on that node. If none of the nodes have free resources, e.g. when all the nodes are busy, the job request is saved in a queue and executed as soon as a node with sufficient resources becomes available.

Tables 1.1 and 1.2 describe all partitions available on the MEGARAPTOR and MICRORAPTOR clusters, respectively. The MEGARAPTOR has partitions `p2, p3` and `p4`. The MICRORAPTOR has only `p1` partition that includes all the nodes included in the cluster.

Slurm accounts

"Slurm account" represents a list of rules and limitations for users included in the account. Many users can share the same Slurm account (i.e., share the same limitations). For instance, all students of the SH2704 course share the `mc` Slurm account. Accounts are guaranteed a specific share of core-hours available on the cluster, but within a single account, users have the same share. Table 2.3 describes all Slurm accounts on the MEGARAPTOR cluster.

Table 2.3: Description of MEGARAPTOR Slurm accounts

Name	Share	TIMEpj	J	NODESpj	CPUSpj	RAMpj	CPUSpu	RAMpu	Partitions
`mc`	9	24 h	inf	1	12	30	160	240	p2-4
`xmc`	4	7 d	1	1	16	50	inf	inf	p3-4
`project`	3	12 h	inf	1	16	30	inf	inf	p2-4, gpu
`xproject`	2	7 d	2	1	16	60	inf	inf	p3-4
`staff`	1	24 h	inf	inf	inf	inf	inf	inf	p3-4, gpu
`xstaff`	1	7 d	2	1	inf	inf	inf	inf	p3-4

Table 2.4 describes all Slurm accounts on the MICRORAPTOR cluster. While MSc students (with accounts `mc` and `xmc`) have certain limitations on resources like CPUs and RAM allocations, their jobs have a brutal priority on both clusters over any other accounts.

Table 2.4: Description of MICRORAPTOR Slurm accounts

Name	Share	TIMEpj	J	NODESpj	CPUSpj	RAMpj	CPUSpu	RAMpu	Partitions
`mc`	10	12 h	inf	1	12	30	30	60	p1
`xmc`	5	2 d	1	1	12	30	inf	inf	p1

In the tables:

- Share represents the resource allocation share. When all users of all the accounts use the cluster as much as possible, then the account with Share of 4 will get twice of the core-hours than users of an account with Share of 2. The values are not normalized. Slurm will sum up all of the defined Share values into Tot and give a specific account the Share/Tot of resources. If a single user, belonging to account `project`, is the only user on MEGARAPTOR then the user will be able to use all CPUs of the entire cluster. Users of the `mc` account will not be able to do that due to the limitation on the number of CPUs and RAM used per user.
- TIMEpj is the maximum time during which a node may be allocated for a job.
- J is the maximum number of running jobs per user.
- NODESpj is the maximum number of nodes that can be allocated per single job.
- CPUSpj is the maximum number of CPUs that can be allocated per single job.
- RAMpj is the maximum amount of RAM in GB that can be allocated per single job.
- CPUSpu is the maximum number of CPUs that can be allocated collectively by all running jobs per user. If the user's running jobs collectively take more CPUs then further submitted jobs will be pending until some of the running jobs complete.

- RAMpu is the maximum amount of RAM in GB that can be allocated collectively by all running jobs per user.
- Partitions are the default partitions, but specific users may get banned from using some of them (read the conditions for using the cluster at the end of this manual).
- The term `inf` mean no limit was set on the specific resource, but that does not always mean that the user can access all available resources since the resources may be constrained by other means.

Priority of submitted jobs
Every ten minutes or so, Slurm adjusts the priority of pending jobs (those queued for later execution). Jobs with a higher priority will be executed before those with a lower priority, even if the lower-priority jobs were submitted earlier. The priority of a specific job reflects several factors.

- The most important factor is the fair share that reflects the "share" value in Table 1.5 and the number of recently completed, running and queued jobs submitted by the specific user. Slurm remembers which users have submitted jobs in the past few days and prioritises jobs submitted by users who have consumed the least resources during that time.
- The priority of pending jobs increases over time. A job that has been pending for a while is more likely to be executed than a job that was just submitted.
- Slurm also prioritises jobs that request the fewest resources, such as CPUs, RAM, and time allocations.

The priority value of each pending job is recalculated frequently based on the above factors. Some jobs may remain pending even when the cluster has available resources. This can occur when all pending jobs require more CPUs or RAM than are available. If a new job is submitted with allocations that do not exceed the currently available resources, Slurm will execute the job immediately. Therefore, it is a good strategy not to request too many resources when the job queue is long.

Essential commands

Command	Purpose
`sinfo`	View the status of partitions (queues) and hardware nodes.
`squeue`	See the list of running and pending jobs.
`sbatch`	Submit a batch script for non-interactive execution.
`srun`	Launch a job step or start an interactive session.
`scancel`	Stop a running or pending job (e.g., scancel 12345).
`scontrol show job`	View detailed information about a specific job.

Anatomy of the sbatch script
To run a calculation, you should create a submission script (usually ending in .sh, but any extension is accepted). This file tells Slurm exactly what resources are required.

Example Script: `my_job.sh`

```
#!/bin/bash
#SBATCH --account=mc              # Slurm account used for the job
#SBATCH --job-name=calc_test      # Name of the job
#SBATCH --output=result_%j.out    # Standard output log (%j inserts JobID)
```

```
#SBATCH --error=result_%j.err     # Error log
#SBATCH --partition=p1            # Which partition to submit to
#SBATCH --nodes=1                 # Number of physical nodes
#SBATCH --ntasks=1                # Total number of tasks
#SBATCH --cpus-per-task=4         # How many logical CPU cores per task you want to block or
#SBATCH --mem=8G                  # Total memory (RAM) required
#SBATCH --time=02:00:00           # Max runtime (HH:MM:SS)

# --- Commands to execute ---
module load python
srun python my_script.py
```

The bottom part of the sbatch file contains commands that will be executed on the computing nodes. These commands usually include loading the Lmod modules, Spack packages or environments, or Apptainer, unless you wish to run a program installed in your own account. The top part of the sbatch file contains parameters that specify the required resources. Below is a description of the most useful parameters:

Job identification & logging:

`--account=<name>` sets a name for the Slurm account used for the job (a user may have several Slurm account with different limitations).

`--job-name=<name>` sets a name for the job (visible in squeue).

`--output=<filename>` directs the "standard output" (what you would normally see on screen) to a file.

`--error=<filename>` directs "standard error" (error messages) to a separate file.

`--mail-user=<email>` sets the email address to receive notifications.

`--mail-type=<type>` defines when to email the user. Options: BEGIN, END, FAIL, REQUEUE, or ALL.

`--comment="text"` adds a custom string/metadata to the job for accounting purposes.

Resource allocation (CPU & RAM):

`--nodes=<min>[-max]` requests a specific number of nodes. (Users of the `m1-4` partitions can skip this parameter since it is fixed to 1.)

`--ntasks=<number>` defines how many MPI processes should be run in parallel. (Users of the `m1-4` partition can skip this parameter since MPI is not allowed on these partition.)

`--cpus-per-task=<number>` specifies how many CPU cores should be allocated to each task. This blocks these CPUs for your task, meaning that other users will not be able to use them. This is relevant for multithreading (e.g. OpenMP or the Python multiprocessing library); however, you must set up your calculation to use the OpenMP library and set it up for using the specific number of CPUs, otherwise your task will only use one CPU thread. Simply allocating (blocking) many CPUs does not mean that your calculation will automatically use them. You must run your calculation in OpenMP mode to utilise the CPUs. Blocking more CPUs than you use on the node violates the rules of this cluster and will result in your account being deactivated.

`--mem=<size>` specifies the memory **per node** (e.g., `--mem=16G`) that Slurm will allocate (block) **for each of your tasks** in the job. If you allocate (block) significantly more RAM than you task needs then you violated the rules of this cluster and your account will be blocked.

`--mem-per-cpu=<size>` specifies memory requested for each allocated CPU coew. Sometimes preferred over `--mem` in high-throughput environments (but it is relevant rather for MPI jobs where the memory demands scale with the number of CPU cores).

`--ntasks-per-node=<number>` forces Slurm to distribute tasks across nodes in a specific way (e.g., 8 tasks on every node).

Scheduling & priority:

`--partition=<name>` specifies the partition to use.
`--time=<days-hours:minutes:seconds>` sets the "Wall clock" limit. If the job hits this limit, Slurm kills it. If you set a lower limit (which may help Slurm to start the job earlier), make sure the limit is large enough for the calculation to complete.
`--dependency=<state:jobid>` tells Slurm not to start this job until another job has finished (e.g., `--dependency=afterok:12345`).
`--begin=<time>` delay the job start until a specific time (e.g., `16:00` or `tomorrow`).

Specialized hardware & placement:

`--gres=<resource>` requests generic resources. Most commonly used for GPUs (e.g., `--gres=gpu:2` for two GPUs).
`--constraint=<feature>` restricts the job to nodes with specific hardware features (e.g., `--constraint="intel"` or `--constraint="avx512"`).
`--exclude=<node_list>` explicitly tells Slurm NOT to use certain nodes.
`--nodelist=<node_list>` explicitly tells Slurm to use nodes from the list. E.g., `--nodelist=n40,n42`.

Advanced behavior:

`--requeue` specifies that the job should be put back in the queue if the node fails.
`--get-user-env` tells Slurm to load your `.bashrc` or login environment on the compute node.
`--array=<indexes>` submits a "Job Array"—running the same script multiple times with different input values (e.g., `--array=1-100`). This parameter creates multiple, independent jobs, each with memory, CPUs and time allocations stated in the sbatch script. These resource allocations are thus not shared by all the created jobs. Each created job gets these resource allocations independently.

Pro-Tip: using `%j` and `%a`

`%j` becomes the Job ID (e.g., `12345`) in `--output` or `--error` files.
`%a` becomes the Task ID (essential for Job Arrays).

2.3 Modules

On HPC clusters, the same version of a program may be required with different features. For example, it may be needed with or without MPI support, or with dependence on a different BLAS library provider. A common Linux system file structure does not directly support such an arrangement. Moreover, system files are usually unavailable in NFC directories, which are needed for deployment on computing nodes. This problem can be solved in various ways. One common approach is to set up an environment module system (e.g. via the Lmod program) within an NFC directory. Lmod (Lua-based Modules) solves this issue by enabling you to modify your shell environment dynamically. When you 'load' a module, Lmod automatically updates your system variables (such as `PATH` and `LD_LIBRARY_PATH`) so that your shell knows exactly which version of the software to run. A 'module' is thus a small file containing the necessary paths and information about dependencies.

Essential commands:
You can view all modules available on the cluster by using the command

```
module avail
```

You can load a module using the command

```
module load <name>
```

You can view all the modules that are currently loaded using the command

```
module list
```

You can remove a specific software package from your environment by the command

```
module unload <name>
```

You can also unload everything at once. This is a great "reset button" if things get messy:

```
module purge
```

You can see exactly what environment variables a module changes using the command

```
module show <name>
```

2.4 Spack

Think of Spack as an 'App Store' for supercomputers. Unlike Lmod, it doesn't just load software; it knows how to build it from scratch with the exact versions and compilers you want. It can also download the source code from an online repository. Spack can use specific compilers (e.g. `gcc@12.1.0`) or build software with particular features. Spack can also create 'Spack Environments' (similar to Conda environments) to keep different projects separate. Loading a Spack environment allows you to load multiple packages at once.

Essential commands:
You can view all the packages that have already been installed on the cluster by using the command

```
spack find
```

You can view all the available versions and optional features (variants) for a package in the online repository using the command

```
spack info <name>
```

The process of loading a program in Spack is similar to that of Lmod. You can load a package by entering the command

```
spack load <package>
```

You can view which Spack packages are currently active in your shell using the command

```
spack find --loaded
```

You can remove a package from your session using the command

```
spack unload <package>
```

You can activate a Spack environment using the command

```
spack env activate <env_name>
```

You can deactivate an active Spack environment using the command

```
spack env deactivate
```

You can view the status (which environment is loaded) using the command

```
spack env status
```

2.5 Conda

The Python-based packages are installed via the Conda environments. The MEGARAPTOR has the following Conda environments:

- The `py314` environment contains Python 3.14 and common Python packages, like: `Numpy`, `Scipy`, `Pandas`, `Polars`, `Sympy`, `Matplotlib`, `Plotly` and `Scikit-Learn`. It also includes the `sage` package (which is the SageMath solver).
- The `py312` environment contains Python 3.12 and the major packages for machine learning applications, like: `PyTorch`, `TensorFlow`, `JAX`, `Keras`, `Scikit-Learn` and other relevant Python packages. These packages are going to run on CPU only, even if you execute them on the `gpu` partition.
- The `py312-gpu` environment contains all the packages that the `py312` environment does, and it also includes the Cuda support for `PyTorch`, `TensorFlow` and `JAX`, and these packages therefore can run on the GPU on the `n01` node.

The MICRORAPTOR contains the `py314` and `py312` Conda environments described above.

To load any of the environment, e.g. `py312`, include these lines in your `sbatch` script file (above your `srun` line):

```
source /share/conda/etc/profile.d/conda.sh
conda activate py312
```

If you want to use the packages in the Conda environment on the master node for data processing then you can simply execute the two above commands on the master node. When you no longer need the packages, you can deactivate the environment on the master node by the command

```
conda deactivate
```

After you source the `/share/conda/etc/profile.d/conda.sh` file, you can list all Conda environments by the command

```
conda env list
```

and you can list all packages included in a specific environment, e.g. `py312` by the command

```
conda list -n py312
```

2.6 Apptainer

The clusters can also run Apptainer (formerly Singularity) containers. While Lmod, Spack and Conda manage software installed natively on the cluster, Apptainer allows you to run software in "containers." Apart from the main program, containers contain the entire operating system (OS), excluding the kernel and drivers. Containers are therefore often quite large (several GB). The host machine's (cluster's) OS kernel must be compatible with the OS in the container. Containers with the Windows OS will not run on a Linux host machine. For this reason, our clusters can only work with containers based on Linux. The main advantage of containers is that they include the exact versions of the libraries required by the main program, and they do not require any libraries on the host machine. However, programs that are natively compiled and installed may run slightly faster than programs in containers, although the difference is often very small.

Apptainer is a container platform designed specifically for HPC. If you are familiar with Docker, you could consider Apptainer its research-focused counterpart. Docker is widely used on single-user workstations, but requires 'root' privileges to run, which poses a significant security risk in a shared cluster environment. Apptainer was designed with a different philosophy. Users have the same permissions and access to files inside and outside the containers. Apptainer can convert and run almost any Docker Hub image (e.g. official TensorFlow or PyTorch) without the need for installation. Apptainer will usually mount your `/users/<username>` home directory in the container by default. However, you will not see any other files (present on the cluster) inside the container.

Essential commands:
In the cluster, if you want to use a container on a computing node, you first need to load the Apptainer using the command

```
spack load apptainer
```

Apptainer images (containers) are stored as single .sif files. You can drop into an interactive terminal inside the container (great for testing) using the command

```
apptainer shell <image.sif>
```

To exit the container shell, press `CTRL+D` or execute the `exit` command. To run a program in the container on the computing node, in the sbatch script, load the Apptainer via Spack and include the command

```
apptainer exec <image.sif> <command>
```

If you are using containers that support the CUDA library on NVIDIA GPU cards, such as the RTX 4090 card on the `gpu` partition, you must instruct Apptainer to pass the NVIDIA drivers to the container by using the `--nv` flag. For example:

```
apptainer exec --nv tensorflow.sif python train_model.py
```

Although Apptainer itself does not host an official image library, it supports pulling from several sources. Most scientific and AI containers (such as those for TensorFlow) are hosted on Docker Hub or NVIDIA NGC. Using the pull command, Apptainer can retrieve the Docker container image and convert it to a .sif file, e.g.:

```
apptainer pull tensorflow.sif docker://tensorflow/tensorflow:latest-gpu
```

You can use any container in the `/share/containers` directory. You can also download and run other containers from your home directory.

3 Computational software

There is no reason for any individual to have a computer in his home.
Ken Olsen, President, Digital Equipment, 1977

3.1 Monte Carlo solvers

3.1.1 Serpent (MEGARAPTOR, MICRORAPTOR)

Serpent is a continuous-energy **Monte Carlo particle transport code** developed by the VTT Technical Research Centre of Finland, widely recognised for its high computational efficiency and specialised focus on reactor physics. Serpent supports **burnup calculations** and complex lattice physics problems. Its typical application is the generation of **homogenised group constants** for deterministic full-core simulators. Serpent is extensively used for 3D spatial homogenisation, research reactor modelling, and safety analysis of advanced designs like **Small Modular Reactors (SMRs)** and Gen-IV concepts.

The cluster supports two versions of the code, Serpent 1 and Serpent 2. Serpent 1.1.18 is installed natively on the cluster, and can be loaded via the module serpent/1.1.18, as

```
module load serpent/1.1.18
```

Then, Serpent 2.1.31 can be loaded as

```
module load serpent/2.1.31
```

When the `serpent module` is loaded, the Serpent code is activated, as well as the `serpentTools` Python environment. SerpentTools is a suite of parsers designed to simplify interaction with Serpent output files. In your Python file, simply include the line `import serpentTools`.

Note on Serpent 1 and Serpent 2

- Both versions of the code, Serpent 1 and Serpent 2, use the same data libraries. Input files for Serpent 1 can be executed with Serpent 2. Execute Serpent 1 and Serpent 2 by the `sss` and `sss2` commands, resp.
- Note that Serpent 2 is approximately twice as slow as Serpent 1 when run as a single process. Therefore, only use Serpent 2 when you need features that are unavailable in Serpent 1. You will get results much faster with Serpent 1.
- Serpent 2 can be executed in parallel mode (using OpenMP) on a single node, which can speed up your calculations. It can utilise multiple CPUs for a single criticality calculation.

- Serpent 1 is compiled on our servers without the option of executing in parallel. This is because Serpent 1 can only use the MPI parallel library, which is not suitable for a single node with shared RAM. However, you can still run multiple Serpent 1 processes simultaneously for different input files.
- KTH currently has a licence for Serpent 2.1.31, but not for Serpent 2.2.x. If you require features exclusive to Serpent 2.2.x, you can request a complimentary licence from NEA and place the executable file in your home directory. You will still need to use the cross-section libraries in `/xsdata`.
- Note that burnup calculations allocate more RAM with each new simulated time step, as new nuclides (e.g. fission products) are added to the simulation. To address this issue, severely limit the number of nuclides simulated in burn-up simulations. See the Serpent manual for details on how to do this. If your simulation requires more RAM than requested from Slurm, it will be terminated.

3.1.2 OpenMC (MEGARAPTOR, MICRORAPTOR)

OpenMC is a modern, open-source Monte Carlo particle transport code originally developed at MIT, distinguished by its focus on high-performance computing and its extensive **Python API** for simulation configuration and data analysis. Unlike many legacy codes that rely strictly on rigid text input files, OpenMC allows users to define geometry, materials, and tallies programmatically, making it highly adaptable for complex workflows. Its typical applications range from **criticality safety assessments** and high-fidelity **reactor physics analysis** (including burnup) to **radiation shielding** and fusion neutronics. Because of its open architecture and support for CAD-based geometry (via DAGMC), it is particularly favored in academic and research settings for developing **coupled multi-physics simulations** where neutronics must interact with thermal-hydraulic or structural mechanics solvers.

The OpenMC code and its Python API are installed natively on the cluster as an Spack package and can be loaded as

```
spack env activate -p omc_env
```

3.1.3 Geant4 (MEGARAPTOR)

Geant4 is a powerful, object-oriented C++ toolkit designed for the high-fidelity Monte Carlo simulation of particles as they pass through and interact with matter. Developed and maintained by a global collaboration led by CERN, it provides a comprehensive suite of physics models covering electromagnetic, hadronic, and optical interactions across an enormous energy range—from a few electron-volts up to the tera-electron-volt scale. Its typical applications are found at the heart of high-energy physics experiments, such as those at the Large Hadron Collider (LHC), where it is used to design and calibrate detectors like ATLAS and CMS. Beyond particle physics, it is the gold standard for medical physics (simulating radiotherapy and PET imaging), space science (evaluating radiation effects on satellites and astronauts), and radiation shielding analysis. On our cluster, Geant4 can be loaded by activating the `geant_env` Spack environment:

```
spack env activate -p geant_env
```

3.2 Cross section processors

3.2.1 NJOY (MEGARAPTOR, MICRORAPTOR)

NJOY is a nuclear data processing code developed at Los Alamos National Laboratory that serves as the essential bridge between evaluated nuclear data libraries and radiation transport codes. Its primary function is to convert raw evaluated data, such as ENDF/B or JEFF, into the ACE (A Compact ENDF) format, which is the standard cross-section input for Monte Carlo codes like MCNP and Serpent. A critical capability of NJOY is its ability to process these libraries at specific temperatures; since raw evaluations typically provide data only at 0 Kelvin, NJOY applies Doppler broadening algorithms to generate pointwise cross-sections that accurately reflect the thermal motion of target nuclei at the exact temperatures required for reactor physics analysis. NJOY is in a container in `/share/containers`, and can be executed via Apptainer.

3.3 Thermal-hydraulics solvers

3.3.1 ABACAS (MEGARAPTOR, MICRORAPTOR)

ABACAS is a system of tools, written by Prof. Henryk Anglart (KTH), to solve flow and heat transfer problems in nuclear reactor systems, primarily in fuel rod assemblies and cores. The system uses the object-oriented approach in which users can build their models by employing objects. The main classes included in ABACAS are **Duct**, which is an *abstract class* to define flow channels, **Hsource**, which is an *abstract class* to define heat sources applied to flow channels, and **Network**, which is an *abstract class* to define interconnected components such as **Ducts**. In addition to these basic classes, ABACAS also has a number of auxiliary classes such as **Xsection**, **Vertex**, and **Vector**. For developers and advanced users, such classes as **Mesh**, **Face**, **Cell**, **Field**, and **Variable** are provided.

To use ABACAS, include these lines in your Slurm script:

```
module load abacas
```

You can learn using ABACAS from Jupyter files in `/share/apps/abacas/0.1.1/doc`.

3.3.2 OpenFOAM (MEGARAPTOR, MICRORAPTOR)

OpenFOAM (Open Source Field Operation and Manipulation) is a highly versatile, open-source C++ framework primarily used for Computational Fluid Dynamics (CFD) and complex multi-physics simulations. Unlike commercial solvers with "black-box" routines, OpenFOAM provides users with an extensive collection of libraries to solve everything from turbulent flows and heat transfer to solid mechanics and electromagnetics. Its typical applications are found in industries such as aerospace, automotive, and energy, where it is used to simulate thermal-hydraulics, combustion, and chemical reactions. In the context of nuclear engineering, OpenFOAM can be coupled with neutronics codes like OpenMC or Serpent to perform high-fidelity multi-physics analysis, allowing researchers to study how coolant flow and temperature distributions affect reactor power and safety margins. OpenFOAM can be loaded as

```
spack load openfoam
```

3.4 Compilers and interpreters

3.4.1 Common Python libraries (MEGARAPTOR, MICRORAPTOR)

Both clusters have many Python packages installed via the Conda manager in a number of Conda environments. See the above Conda Section for details on the list of packages and how to load them. You can load Python 3.14 packages by executing the commands

```
source /share/conda/etc/profile.d/conda.sh
conda activate py314
```

Apart from the Conda packages, the cluster also contains some Python packages installed via Spack. See the list of all Spack packages via the command

```
spack find
```

Some Spack packages appear to be present multiple-times, even in the same version. (They differ in their dependency packages). In that case, they will differ by their hash sign, and you will need to use the hash sign to load one of them. Spack will give you a hint how to do that. You can load various Spack packages simultaneously.

If you need a Python library that is not installed on our cluster, you can either request the manager to add it among the Spack or Conda packages, or you can install them in your own account via your own Miniconda installation.

3.4.2 Julia, SageMath, R, Octave (MEGARAPTOR, MICRORAPTOR)

Julia is a high-level, high-performance programming language combining readability of Python or MATLAB with the execution speed of C or Fortran. Its unique architecture relies on Just-In-Time (JIT) compilation via LLVM and a powerful multiple dispatch paradigm, which allows the compiler to generate highly optimised machine code based on the specific types of function arguments. Its typical applications are centred on high-performance scientific computing. Julia is a rising star in Scientific Machine Learning (SciML), as its native differentiability and speed make it ideal for training neural networks that are physically constrained by differential equations.

You can activate Julia by the command:

```
spack load julia
```

SageMath is a comprehensive, open-source computer algebra system designed to be a viable free alternative to proprietary software like Mathematica, Maple, and MATLAB. It is built as a unifying layer on top of nearly 100 established scientific libraries—including NumPy, SciPy, Maxima, and R—all accessible through a single, consistent Python-based interface. Its typical applications cover a massive range of pure and applied mathematics, such as number theory, cryptography, group theory, and symbolic calculus.

You can activate SageMath by loading the `py314` Conda environment (see the Conda Section).

R is a powerful programming language and environment specifically designed for statistical computing and data visualisation. Originally derived from the S language, R has evolved into

an essential tool for data scientists and researchers due to its vast ecosystem of over 18,000 packages hosted on the Comprehensive R Archive Network (CRAN). Its architecture is centred around vector-based operations, making it exceptionally efficient for handling large matrices and complex datasets. The typical application for R involves advanced statistical modelling, exploratory data analysis, and the creation of publication-quality graphics using libraries like ggplot2.

The **R** interpreter can be loaded by the command

```
spack load r
```

GNU Octave is a high-level, interpreted programming language primarily intended for numerical computations, serving as the most prominent free and open-source alternative to MATLAB. It features a powerful mathematics-oriented syntax with built-in support for multidimensional arrays and matrices, which are treated as fundamental data types. Its typical applications include solving linear and nonlinear problems, performing statistical analysis, and executing large-scale numerical experiments in fields such as signal processing, control theory, and fluid dynamics. Because its syntax is largely compatible with MATLAB, it is frequently used in academia and engineering for rapid prototyping and teaching, allowing researchers to develop complex mathematical models and visualise data with 2D and 3D plotting tools without the need for expensive proprietary licenses.

GNU Octave interpreter can be loaded by the command

```
spack load octave
```

3.5 Machine Learning tools

You can load all the following tools via the `py312` or `py312-gpu` Conda environments:

```
source /share/conda/etc/profile.d/conda.sh
conda activate py312
```

3.5.1 TensorFlow

TensorFlow is an industrial-grade open-source platform for machine learning and deep learning developed by the Google Brain team. It is built around a flexible ecosystem of tools, libraries, and community resources that allow developers to design, build, and deploy complex neural networks across a variety of hardware, from local CPUs and GPUs to massive TPU clusters in the cloud.

3.5.2 PyTorch

PyTorch is a versatile, open-source machine learning framework developed primarily by Meta's AI Research lab (FAIR) that has become the preferred choice for the global research community due to its imperative programming style and "Pythonic" nature. Additionally, its tight integration with Python's scientific stack (NumPy, SciPy) and its native support for GPU acceleration make it a convenient tool for rapid prototyping and academic experimentation in deep learning.

Apart from loading PyTorch via Conda, you can also load PyTorch as:

```
spack load py-torch
```

You can also load Keras with PyTorch backend as:

```
spack load py-keras
```

3.5.3 JAX

JAX is a high-performance numerical computing library developed by Google that combines the familiar syntax of NumPy with the power of hardware acceleration and advanced functional programming transformations. Its typical applications go beyond standard deep learning to include differentiable programming, scientific simulations, and probabilistic programming, where researchers need to compute high-order derivatives or solve complex physical equations with massive parallelisation.

Apart from loading JAX via Conda, you can also load JAX on MEGARAPTOR as

```
spack load py-jax
```

3.5.4 Scikit-Learn

Scikit-Learn is a premier open-source machine learning library for Python, built on top of NumPy, SciPy, and matplotlib to provide a robust and user-friendly toolkit for classical statistical modelling. Unlike deep learning frameworks that focus on neural networks, Scikit-Learn excels at "traditional" machine learning algorithms, offering highly optimised implementations for classification, regression, clustering, and dimensionality reduction. It is widely celebrated for its consistent API design, where almost every model follows the same "fit-predict" workflow, making it incredibly easy to swap algorithms during experimentation.

Apart from loading Scikit-Learn via Conda, you can activate Scikit-Learn as

```
spack load py-scikit-learn
```

4 User accounts

People disagree with me. I just ignore them.
Linus Torvalds, regarding the use of C++ for the Linux kernel.

4.1 Secure access

Usually, logging into a remote server requires a username and password. However, passwords can be guessed, stolen, or cracked by bots. Therefore, our clusters require you to log in using an SSH key, a more secure alternative. They will not normally accept a password login.

SSH keys can be analogised to physical keys and locks. Uploading a "public SSH key" to a remote server corresponds to installing your own lock on a door that you want to be able to open whenever you like. A "private SSH key" corresponds to the physical key that unlocks the door. You can generate both the public and private SSH keys on your laptop so that they work together, and then upload the public key to the remote server. When your laptop accesses the remote server, it shows its private SSH key. The server then checks if it can "unlock" the public key on your account.

Request the account from the manager. In the email, attach you public ssh key that you can generate according to the procedure below.

Step 1: generate your key pair
No matter your operating system, the process is now almost identical because Windows, MacOS, and Linux all include the `ssh-keygen` tool by default.

1. Open your **Terminal** (Mac/Linux) or **PowerShell/Command Prompt** (Windows).
2. Type the following command and press **Enter**:
   `ssh-keygen -t ed25519 -C <your_email@kth.se>`
   (Note: Ed25519 is the modern standard for security and speed.)
3. **Prompt - "Enter file in which to save the key":** Press **Enter** to accept the default location.
4. **Prompt - "Enter passphrase":** This is an optional password for your key file. You can leave it blank (for passwordless access to the cluster) and press **Enter**.

You now have two files in a hidden folder called `.ssh` in your user directory:

- `id_ed25519` (Your **Private** key — Keep this secret!)
- `id_ed25519.pub` (Your **Public** key — This is what you will send to the manager.)

Send the public key to the manager from your KTH email account to set up your account. If you send a request from another server then the manager will not fix an account for you. Once you have access, you can add more public keys to the file `~/.ssh/authorized_keys`, which will allow you to log in from different computers.

- **Add your key to the "Authorized" list**:
  Run the following command, but replace `PASTE_YOUR_KEY_HERE` with the complete string you find in the file id_ed25519.pub:

  ```
  echo "PASTE_YOUR_KEY_HERE" >> ~/.ssh/authorized_keys
  ```

  e.g.,

  ```
  echo "ssh-ed25519 AAAAC3Nz...O2ogjVu7 jandufek@kth.se" >> ~/.ssh/authorized_keys
  ```

  (You don't need to worry that somebody could misuse your public key since nobody can access your cluster account using a public key that is installed in your account. It is perfectly fine to send it via an email that somebody else could possibly read. The private key is necessary for that.)

- **Set permissions**:

  ```
  chmod 600 ~/.ssh/authorized_keys
  ```

Beware that if your attempt to log in fails fourtimes during 24h then the cluster will ban your IP address and reject further connection attempts from this address permanently (contact the manager if this happens to you and provide the IP address).

4.2 Request an account

Students registered in Nuclear Engineering programmes at KTH can get an account on MICRO-RAPTOR upon email request to the manager. You must send the request from your KTH email account, and you need to attach the public SSH key to the email. Students of the Monte Carlo course will get group accounts on the MEGARAPTOR automatically after submitting their public SSH key via Canvas.

4.3 Logging in the cluster

From Mac/Linux terminal or Windows PowerShell terminal, type:

```
ssh -p <port_number> <username>@<ip_address>
```

where `<port_number>`, `<username>` and `<ip_address>` will be given to you by the manager.

4.4 Practical info

Our clusters run on Linux, so you will need to familiarise yourself with the Linux operating system. This involves transferring files from the server account to your computer via the SFTP protocol, connecting to the server from your computer via the SSH protocol, managing files on your account, creating, editing and processing files, and launching calculations on the server.

If you are not familiar with Linux terminal, consider the following links for guidance:

- Linux command line for beginners
- Basic Linux commands

- Nano (command-line text editor)
- Using top and htop
- Midnight Commander - the visual file manager for Linux terminal.

You should also become familiar with **Tmux** (Terminal Multiplexer) that is an essential productivity tool for developers and researchers working in a command-line environment. Its most powerful feature is session persistence: you can detach from a running session and log out of a remote supercomputer or server, only to reattach later (even from another machine) while all processes continue undisturbed. If you execute a process without Tmux (or another multiplexer) then the process would terminate as soon as your connection to the server is interrupted. You can learn Tmux here.

You will also need to transfer files between your PC and the cluster. For that purpose, you can use e.g. Termius, which also includes an SSH client, FileZilla, or you can mount your cluster home directory using `sshfs` as an external drive in your PC. Here is a quick tutorial on how to set up sshfs connection on your PC with MacOS or Linux:

1. Install `sshfs` locally on your PC.
2. On your PC, create a directory for the remote disk, e.g. `~/megaraptor` where `~` is the path to your home directory.
3. Mount the remote drive:

   ```
   sshfs accountname@130.237.70.232:/users/<username> ~/megaraptor
   ```

   The remote disk should be visible now in your PC, and you can use it just like any other disk. You can edit files with editors installed on your PC.
4. If you like, during the mounting, you can optimise the performance of the remote disk using various options:

   ```
   sshfs accountname@130.237.70.232:/users/<username> ~/megaraptor \
     -o reconnect \
     -o ServerAliveInterval=15 \
     -o ServerAliveCountMax=3 \
     -o cache=yes \
     -o kernel_cache \
     -o compression=no \
     -o allow_other
   ```
5. You can unmount the disk by

   ```
   usermount -u ~/megaraptor
   ```

It is also possible to mount your cluster home directory using `sshfs` as an external drive in your PC with Windows. Download the latest version of Winfsp and SSHFS-win. In your Windows file explorer, righ-click on 'this PC' and select 'Map network drive'. Choose any available destination on your PC, and enter the address of the remote folder you want to mount as:

```
\\sshfs\<username>@130.237.70.232
```

Then, enter your remote account password in the dialog window that shows up. Now, your remote directory should be visible in your file explorer.

5 How to edit and debug input files

> *Debugging is twice as hard as writing the code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it.*
> Brian Kernighan

5.1 Preparing Serpent input files

The Serpent input file is an ASCII text file with whatever extension you like (or without any extension). It may be easiest for you to prepare the file on your laptop and copy it to the server using

- FileZilla,
- scp (a terminal command),
- other file managers with remote access feature.

Example for Group 1:

```
scp my_input.inp MC25G1@130.237.70.232:~
```

5.2 Converting the input file format to Linux

Once you have connected to the master node and copied the input file to your folder, you need to ensure that the ASCII input file is in the correct format. This is because ASCII files have slightly different formats on Linux, Mac and Windows. These operating systems use different end-of-line characters in ASCII files. If you create an ASCII input file on your Windows laptop, Serpent may not interpret your input correctly and may halt. You can use the `dos2unix` command to convert an ASCII file created on Windows into a Linux-compatible format. Use `mac2unix`, `unix2dos` and `unix2mac` for other conversions. To convert a file created in Windows, type:

```
dos2unix input.inp
```

5.3 Manage your files on the cluster

You can create any folder structure within your account and manipulate files within those folders. You can use the Midnight Commander (type `mc`) to move or copy files and create or delete folders. The Midnight Commander also includes a simple ASCII file editor. Press F4 to open the editor. The first time you open the editor, you can choose whether to use the built-in editor or another terminal editor. It may be easier to make simple modifications to your input files directly on the master node than on your laptop and then upload new revisions.

5.4 Debugging Serpent input file on the master node

If you want to check whether Serpent can run a simulation with your input file, then on the master node type:

```
sss <input_file>
```

or

```
sss2 <input_file>
```

to execute your calculation with Serpent 1 or Serpent 2, resp.

You will see error messages in the terminal window. Some bugs are easy to fix, while others are more challenging. Always try to plot the geometry first. As long as there are errors in the geometry, your simulation will not produce the correct results, even if it starts.

Once all the bugs have been eliminated, the calculation will start on the master node and successive cycles will start to be completed. At this point, however, you must terminate your simulation by pressing CTRL+C, since no simulations are permitted on the master node. Instead, you must submit a Slurm request to run your simulation on a computing node.

6 How to run simulations on compute nodes

```
/*
Dear Maintainer
When I wrote this code, only I and God  understood what it was doing.
Now, God only knows!
Once you are done trying to 'optimize' this routine,
and you have realized what a terrible mistake that was,
please increment the following counter as a warning
to the next guy.

total_hours_wasted_here = 42
*/
```

6.1 Which Slurm account you have

You can check which Slurm accounts are available to you using the command:

```
sacctmgr show assoc user=$USER format=Account
```

6.2 Estimation of RAM allocation

In your sbatch script, you can allocate (block) the amount of RAM for your calculation. If you block a large amount of RAM then this will limit the RAM resources for other users of the cluster and the node may become largely idle due to that. If you allocate a smaller amount than your calculation needs then your calculation may crash. Therefore it is very important that you make a very good estimation. To do this, run a short test simulation on the master node using the `time` command:

```
/usr/bin/time -v sss <input_file>
```

or

```
/usr/bin/time -v sss2 <input_file>
```

depending whether you plan to run Serpent 1 or Serpent 2 simulation. Importantly, use the same number of neutrons per cycle as you intend to use on the node, but drastically reduce the number of cycles for this test, so that the test calculation finishes quickly. Once the calculation has finished, you will find additional information at the end of the terminal output. One of the lines will look like this:

```
Maximum resident set size (kbytes): xxxxx
```

where `xxxxx` is the amount of RAM in kB that your calculation took. We can denote this value in GB as M_{test}, i.e. $M_{\text{test}} = \text{xxxxx}/10^6$. If you use OpenMP on the computing node, the memory demands will increase. I recommend estimating the RAM allocated for criticality calculations in GB using the formula

$$\text{Requested RAM} = 1.25 \times M_{\text{test}} \times (1 + 0.08 \times (N_{\text{cores}} - 1))$$

where N_{cores} is the number of CPU cores you want to use for the criticality calculation (via OpenMP).

If you run a burnup simulation, the allocated memory will increase over time as more fission products are added to the fuel. Therefore, I recommend severely limiting the number of nuclides that you track during the burnup simulation. Run a quick test calculation with a small number of cycles and a few (but large) time steps that cover the entire time interval, and estimate M_{test} as above. I recommend that you estimate the RAM allocated for the burnup calculation in GB using the formula

$$\text{Requested RAM} = 1.8 \times M_{\text{test}} \times (1 + 0.08 \times (N_{\text{cores}} - 1))$$

If you don't allocate any RAM via the sbatch script (by not using the `--mem` flag) then Slurm will allocate 1 GB RAM per each CPU that you use. That may not be sufficient for your calculation, so please always allocate the memory by yourself.

6.3 Running a simulation in the parallel mode

Running a simulation in parallel mode can reduce the wall-clock run-time of the simulation. While the Slurm partitions available to MSc students do not permit the execution of MPI simulations across multiple nodes (using OpenMPI or MPICH libraries), they do allow multi-threaded simulations to be run inside the nodes.

By specifying the sbatch script line

```
#SBATCH --cpus-per-task=6
```

you can allocate (block) a certain number of CPU cores for your calculation. In the example above, 6 CPU cores are allocated on a single node. **This allocation alone does not cause your simulation to run in parallel, multi-threaded mode.** In order to use the allocated cores, you must satisfy the following two conditions:

1. The code you intend to use must be able to utilise multiple CPU cores; for example, via the OpenMP library. If the code does not have this capability, it will use only one CPU core, regardless of how many cores you allocate on the node for your calculation.
2. You must instruct the code to use the allocated CPU cores. See the Serpent 2 example below for guidance on how to do this.

If the above two conditions are not met, your calculation will use a single CPU core, even if you have allocated more CPU cores to your calculation. This would violate the server's rules and result in your account being blocked. Please note that Serpent 1 cannot use multiple cores. Therefore, allocating more than one CPU core to your Serpent 1 simulation will only result in your account being deactivated!

The reduction of the wall-clock time of the execution of a parallel calculation usually does not scale optimally with the number of CPU cores (employing e.g. 10 CPU cores will not cut the execution time to one-tenth). This is because only certain parts of a Serpent 2 simulation can be parallelised. At the end of each criticality cycle, the simulation will switch out of parallel mode and run certain procedures on a single CPU core.

The efficiency of parallel computing can be described using Amdahl's law. The overall performance gain is limited by the fraction of the code that cannot be parallelised. Furthermore, the overall CPU performance is limited by RAM memory bandwidth (i.e. the rate at which the CPU can read or write data to memory) in applications that frequently access RAM, such as Serpent 2 and other MC codes. For these two reasons, using more than about 10 CPU cores for your Serpent 2 (or OpenMC) simulations will not meaningfully reduce computing time. **For this reason, `m1-4` partitions have a max limit on the number of CPUs per job. You are not allowed to use more CPUs than given by this limit. Check the specification of the partitions in Table 1.4.**

6.4 Examples

You can run a simulation on the compute node using the sbatch command. All Slurm output files and Serpent output files will appear in the same folder where you run sbatch, irrespective of which node the simulations run on. Here are examples for specific cases (you will find them in the folder `examples` in your home account on the cluster):

You can copy the following examples from `/share/examples.tar` as:

```
cp /share/examples.tar ~
tar -xf ~/examples.tar
```

Running single-core Serpent 1 simulation
Prepare the script file `single.sh` as:

```
#!/bin/bash
#SBATCH --account=mc
#SBATCH --job-name=bwr_1core
#SBATCH --output=output.txt
#SBATCH --error=errors.txt
#SBATCH --partition=p1
#SBATCH --cpus-per-task=1
#SBATCH --mem=2G
#SBATCH --time=01:00:00

module purge
module load serpent/1.1.18
ulimit -s unlimited
srun sss bwr
```

and execute it as:

```
sbatch single.sh
```

Running Serpent 2 in the parallel-mode
Serpent 2 uses multi-threading (OpenMP). The corresponding script file, named e.g. `parallel.sh`, may look like:

```
#!/bin/bash
#SBATCH --account=mc
#SBATCH --job-name=serp_omp
#SBATCH --output=output.txt
#SBATCH --error=errors.txt
#SBATCH --partition=p1
#SBATCH --cpus-per-task=6
#SBATCH --mem=4G
#SBATCH --time=02:00:00

module purge
module load serpent/2.1.31
ulimit -s unlimited
export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
srun sss2 bwr
```

and execute it as:

```
sbatch parallel.sh
```

Explanation:

- `cpus-per-task=6` tells Slurm you want 6 cores.
- `OMP_NUM_THREADS` tells OpenMP library (used by Serpent 2) how many cores to use (Slurm makes the value of the cpus-per-task parameter available via the `SLURM_CPUS_PER_TASK` value).

Running multiple Serpent 1 simulations on many cores
This method, known in Slurm terminology as "array jobs", is useful when you need to run many different Serpent (1 or 2) independent simulations. Let's prepare a script named e.g. `array.sh`:

```
#!/bin/bash
#SBATCH --account=mc
#SBATCH --job-name=array_serpent
#SBATCH --output=out_%a.txt
#SBATCH --error=errors_%a.txt
#SBATCH --array=1-3
#SBATCH --partition=p1
#SBATCH --cpus-per-task=1
#SBATCH --mem=2G
#SBATCH --time=01:00:00
```

```
module purge
module load serpent/1.1.18
ulimit -s unlimited
srun sss bwr_${SLURM_ARRAY_TASK_ID}
```

Submit:

```
sbatch array.sh
```

This creates three independent simulations running in parallel, each possibly on different nodes or cores. Slurm can utilise all available nodes if you have many array jobs.

6.5 Monitoring your jobs

List all your jobs (example for group `MC25G1`):

```
squeue -u <username>
```

The output will give you a list of running jobs, and you can see `JOBID` (identification number for each job). You can use the `JOBID` to cancel the specific job by the command:

```
scancel <jobid>
```

You can cancel all your jobs:

```
scancel -u <username>
```

The state of jobs given by the `squeue` command is listed in the ST column. The most common job states are:

- R: Running
- PD: Pending
- CG: Completing
- CA: Canceled

You can also monitor all submitted jobs via the command

```
squeue -o "%.18i %.9P %.8j %.8u %.2t %.10M %.6D %.10C %.10m %R"
```

that will also show the number of allocated CPUs and the allocated amount of RAM for each submitted job.

6.6 Checking available nodes

You can check the state of computing nodes with the command

```
sinfo
```

You can list nodes with detailed info on CPU and memory using the command

```
scontrol show nodes
```

7 Terms and conditions for use

If at first you don't succeed, try, try again. Then quit. There's no point in being a damn fool about it.
W. C. Fields

You can use the cluster under the following conditions:

1. You must run a short test calculation on the master node and measure the computing time, as explained in this document. You must then set up parameters of your calculation so that the wall-clock time of your calculation will not accede your time allocation or the time limit set up for your Slurm account. You must assume that Slurm may execute your calculation on the slowest node of the partition and that the node could be under a full stress.
2. You must run a short test calculation on the master node and measure the RAM requirements, as explained in this document. You are not allowed to allocate on the computing nodes more than 1.5-times the measured requirements. Beware that MC burnup calculations will keep increasing RAM demand with each new time step, so your test burnup calculation should cover the entire target burnup time (use very large time steps and very poor statistics to run the test quickly on a single core). To reduce the RAM requirements, limit the number of nuclides for which the burnup MC simulation loads XS during the simulation.
3. At any time, each user account can run maximum a single test calculation (or any other process) on the master node during **up to 90 minutes**, and not any longer. Before starting the test calculation on the master node, you must check (with the top or htop command) that the master node has at least 60 GB of free RAM, and not start the simulation until at least 60 GB RAM becomes available. Apart from running test calculations, you can use the master node for debugging and data processing. When you debug a Serpent input file and it appears to run well in the terminal window on the master node, terminate it with Ctrl+C, and set up a short test run on the master node for the purpose of estimating the time and memory allocations.
4. You must not start any processes on the master node in the parallel mode (OpenMP, MPI or any other). You can run simulations in the parallel mode only on the computing nodes.
5. On the compute node, your calculation must use the same number of CPUs that you allocate via the Slurm sbatch script (using the `--cpus-per-task` parameter).
6. You are not allowed to copy the nuclear data library from `/xsdata` to your home directory. Use the nuclear data library exclusively from the `/xsdata` directory. If you need another library, contact the manager.
7. Students can use the cluster only for tasks assigned by teachers at KTH or for working on problems on interest that directly relate to nuclear engineering. Running any software that has no relation to nuclear engineering will violate the cluster conditions.
8. You must not share your private SSH key (active on the cluster) with anybody.

9. You must not set up any kind of personal server on your account that would run constantly in the background. This includes VS Code server, Cursor server, etc. Any background process associated with your account running longer than 90 minutes (as any regular test calculation) will triger a violation of the cluster conditions.

What happens when you violate any of the above conditions

The cluster is randomly checked for violations of the usage conditions. If you violate the conditions, your jobs (on master or compute nodes) will be terminated, and you will lose access to the best partition you had access to. The manager will leave a warning message in a file in your user account directory. If you violate the conditions multiple times and lose access to all Slurm partitions then your account will be permanently deactivated.

Note: Your account will expire automatically. It is important that you complete your work on the cluster and backup your files before your account expires. It is your responsibility to check the expiration date of your account. Your files will be deleted when your account expires. You can check the expiration date of your account using the command

```
chage -l $USER
```